

SmartBugs - Test Plan

Luciano Bercini

April 2026

Contents

1	Introduction	2
2	System Under Test	2
3	Testing Objectives	2
4	Testing Approach	2
4.1	Unit Testing	3
4.2	Component Testing	3
4.3	Integration Testing	3
4.4	Regression Testing	3
5	Adequacy Criteria	4
6	Testing Tools	4
7	Assessment of the Existing Test Suite	5
8	Features to Test	5
8.1	CR_1: Integration of Feature Extraction Module	5
8.2	CR_2: Machine Learning-Based Tool Selection	5
8.3	CR_3: Integration into SmartBugs Workflow	6
9	Pass and Fail Criteria	6
10	Category Partition	6
10.1	Description	6
10.2	Parameters	6
10.3	Categories	7
10.4	Constraints	7
10.5	Derived Test Frames	8
11	Conclusion	8

1 Introduction

This document defines the test plan for the evolution of the SmartBugs framework.

The project aims to extend SmartBugs by introducing an intelligent tool-selection mechanism based on feature extraction and machine learning models. The modified system should automatically select the most suitable analysis tools for a given smart contract while preserving the behavior of the original SmartBugs workflow.

The purpose of this test plan is to define the testing strategy adopted for the project, including the testing levels, regression testing approach, adequacy criteria, testing tools, features to test, pass/fail criteria, and the category partition used to derive the post-modification test cases.

Detailed test case specifications and execution results are reported in separate testing documents.

2 System Under Test

The system under test is SmartBugs, a framework for the automated analysis of Ethereum smart contracts. The original system allows users to execute one or more vulnerability detection tools on Solidity contracts or bytecode files.

The proposed evolution introduces an intelligent workflow composed of the following steps:

1. feature extraction from the input smart contract;
2. selection of the subset of features required by the machine learning models;
3. execution of pre-trained models;
4. aggregation and filtering of predicted tools;
5. execution of SmartBugs with the selected tools.

The modification is designed to be modular and backward compatible. Therefore, the original manual workflow must remain available after the introduction of the intelligent mode.

3 Testing Objectives

The testing activity has the following objectives:

- verify the correctness of the new feature extraction functionality;
- verify the correct execution of the machine learning-based tool selection process;
- verify that predicted tools are correctly aggregated and filtered;
- verify that the intelligent workflow correctly invokes the existing SmartBugs execution pipeline;
- verify that the command-line options required by the intelligent workflow are parsed and stored correctly;
- ensure that the original SmartBugs behavior is not affected by the modifications;
- provide a baseline for regression testing before and after the implementation.

4 Testing Approach

The testing strategy follows a layered approach, combining unit testing, component testing, integration testing, and regression testing. The goal is to ensure both correctness of the new functionality and preservation of the existing SmartBugs behavior.

At the lowest level, unit tests validate isolated components such as feature selection, model loading, and prediction handling. These tests focus on verifying the correctness of individual functions without external dependencies.

At the component level, tests validate the interaction between closely related modules, such as the integration between feature extraction and model inference, or between tool selection logic and SmartBugs task generation.

At the system level, integration tests aim to validate the full workflow introduced by the intelligent command, including feature extraction, model inference, tool selection, and execution through the SmartBugs pipeline.

Mocking is used selectively to isolate external dependencies such as Docker execution and Solidity compilation, while still preserving realistic interactions between internal modules.

4.1 Unit Testing

Unit testing is used to validate the behavior of individual components in isolation.

In the modified system, unit tests will focus on:

- feature extraction wrapper;
- feature subset selection;
- model loading;
- model prediction;
- tool aggregation and filtering.

The goal of unit testing is to verify that each newly introduced component works correctly before it is integrated with the rest of the system.

4.2 Component Testing

Component testing validates the interaction between closely related modules.

In particular, component testing will focus on:

- interaction between feature extraction and feature selection;
- interaction between feature selection and model inference;
- interaction between model inference and tool aggregation;
- interaction between the tool selection logic and SmartBugs task generation.

This testing level is useful because the proposed change introduces multiple new components that must exchange structured data correctly.

4.3 Integration Testing

Integration testing validates the complete intelligent workflow.

The main integration scenario is:

1. the user provides a smart contract as input;
2. the system extracts features from the contract;
3. the system executes the machine learning models;
4. the system selects the final list of tools;
5. the selected tools are passed to the original SmartBugs pipeline;
6. SmartBugs generates and executes the corresponding analysis tasks.

External dependencies such as Docker execution and Solidity compilation may be mocked where necessary, especially when the goal is to validate the internal coordination logic rather than the behavior of external tools.

4.4 Regression Testing

Regression testing is essential because the project evolves an existing software system.

The proposed modification introduces a new intelligent execution mode, but the original manual workflow must remain unchanged. Therefore, regression testing focuses on verifying that the original SmartBugs functionalities are not affected.

The regression strategy consists of:

- executing the pre-modification test cases again after implementation;
- re-executing the existing unit tests provided in the SmartBugs test suite;
- verifying that file collection still works correctly;
- verifying that tool loading still works correctly;

- verifying that task generation still works correctly;
- verifying that Solidity compiler handling is not affected;
- validating that manual tool selection still behaves as in the original system;
- comparing outputs before and after the modification on representative input contracts.

The existing pytest test suite will be reused as implementation-level evidence for the regression tests.

5 Adequacy Criteria

The adequacy criteria adopted in this project are primarily functional and change-oriented, rather than based on full structural coverage.

A test suite is considered adequate if:

- each change request is covered by at least one test case;
- each newly introduced component is covered by dedicated tests;
- each modified module is tested along the execution paths affected by the proposed changes;
- the complete intelligent workflow is covered by at least one integration test;
- the original SmartBugs workflow is covered by regression tests;
- error cases related to invalid input, missing features, invalid predictions, and empty tool lists are considered;
- the command-line configuration of the intelligent workflow is covered by at least one integration test;
- backward compatibility is explicitly verified by testing scenarios where tools are manually specified by the user.

The goal is not to achieve exhaustive structural coverage, but to cover the functionalities directly affected by the proposed evolution and the existing behaviors that must be preserved.

6 Testing Tools

The project uses the same testing infrastructure already adopted by SmartBugs.

The main testing tools are:

- `pytest`, used to structure and execute tests;
- `pytest-mock`, used to mock external dependencies and isolate components;
- `pytest-cov`, used to collect coverage information;
- `pytest-timeout`, used to prevent tests from hanging indefinitely;
- fixtures defined in `conftest.py`, used to provide reusable test data and environments;
- temporary filesystem utilities provided by `pytest`, such as `tmp_path`.

Mocking utilities such as `unittest.mock` are used selectively to isolate dependencies such as Docker execution, Solidity compiler resolution, and external feature extraction execution.

7 Assessment of the Existing Test Suite

The SmartBugs project already includes a structured test suite based on `pytest`. The tests are primarily organized under the `tests/` directory and cover key components of the system.

In particular, the file `tests/unit/test_smartbugs.py` provides coverage of the orchestration logic, including:

- file collection using various patterns and formats;
- task generation for different tool modes and configurations;
- tool loading and tool aliases;
- handling of edge cases such as missing pragmas, duplicate files, incompatible tools, and result directory collisions;
- validation of the main execution flow.

Additionally, `conftest.py` defines fixtures that support testing through reusable contracts, mocked Docker clients, sample compilation outputs, and temporary file structures.

However, the current test suite is mainly focused on unit and component testing. Although some tests simulate multi-module interactions, they rely on mocking and do not represent full end-to-end execution scenarios.

Furthermore, the original SmartBugs test suite did not include a dedicated integration testing layer for the proposed intelligent workflow.

As a result, while the existing test suite provided a useful foundation for validating individual components and supporting regression testing, it was not sufficient on its own to validate the proposed intelligent workflow in a realistic environment.

For this reason, additional integration tests were introduced as part of the proposed changes. In particular, dedicated integration tests were added to validate:

- the integration between command-line parsing and runtime configuration for the intelligent mode;
- the integration between feature extraction, model loading, prediction aggregation, and intelligent tool selection.

These additions extend the original test suite with explicit support for validating the new intelligent workflow.

8 Features to Test

The features to test are organized according to the proposed change requests.

8.1 CR_1: Integration of Feature Extraction Module

The system must be able to extract structured features from Solidity smart contracts.

The tested features are:

- execution of the feature extraction component;
- production of a structured feature vector;
- handling of invalid Solidity contracts;
- handling of empty or non-analyzable files.

8.2 CR_2: Machine Learning-Based Tool Selection

The system must be able to use pre-trained machine learning models to predict suitable SmartBugs tools.

The tested features are:

- loading of serialized `.joblib` models;
- selection of the required feature subset;
- execution of model predictions;
- handling of invalid or missing model outputs.

8.3 CR_3: Integration into SmartBugs Workflow

The system must integrate the intelligent tool-selection mechanism into the existing SmartBugs pipeline.

The tested features are:

- aggregation of predicted tools;
- removal of duplicate predictions;
- removal of non-informative predictions;
- parsing and storage of the command-line options required to activate the intelligent workflow;
- invocation of SmartBugs using the selected tools;
- preservation of the original manual workflow.

9 Pass and Fail Criteria

Each test case is associated with an oracle describing the expected behavior.

A test is considered **passed** if:

- the observed output matches the expected result;
- the expected error is raised in invalid scenarios;
- the system preserves the original workflow when intelligent mode is disabled;
- no unexpected exception occurs.

A test is considered **failed** if:

- the observed output differs from the expected result;
- an invalid tool is not filtered;
- duplicate tools are not removed;
- the original manual workflow is affected;
- an unexpected runtime error occurs.

10 Category Partition

The Category Partition technique is used to derive representative test cases for the intelligent tool-selection workflow.

10.1 Description

The modified system must automatically select and execute the most appropriate SmartBugs tools based on features extracted from a smart contract and predictions generated by machine learning models.

10.2 Parameters

The main parameters considered for the category partition are:

- input contract;
- extracted features;
- selected feature subset;
- model predictions;
- intelligent workflow configuration;
- selected tools;
- execution mode.

10.3 Categories

Category Name	Category Values
Contract Input	C1: Valid Solidity contract C2: Solidity contract with syntax errors C3: Empty or invalid file
Feature Extraction	F1: Features correctly extracted F2: Partial feature extraction with missing values F3: Feature extraction failure
Feature Selection	S1: All required model features are available S2: Some required model features are missing S3: Feature subset cannot be constructed
Model Prediction	M1: Single valid tool predicted M2: Multiple valid tools predicted M3: Duplicate tools predicted M4: Invalid or unknown tool predicted M5: <code>nessun_tool</code> predicted
Tool Selection	T1: One valid tool selected T2: Multiple valid tools selected T3: Empty tool list after filtering
Execution Mode	E1: Intelligent mode enabled E2: Manual mode enabled
Intelligent Configuration	G1: Valid intelligent workflow configuration G2: Missing intelligent workflow configuration G3: Invalid intelligent workflow configuration

Table 1: Category Partition for the intelligent workflow

10.4 Constraints

The following constraints define valid combinations among categories.

Relation	Constraint
Contract Input $\rightarrow$ Feature Extraction	C1 $\rightarrow$ F1 or F2 C2 $\rightarrow$ F3 C3 $\rightarrow$ F3
Feature Extraction $\rightarrow$ Feature Selection	F1 $\rightarrow$ S1 F2 $\rightarrow$ S2 or S3 F3 $\rightarrow$ no feature selection possible
Feature Selection $\rightarrow$ Model Prediction	S1 $\rightarrow$ M1, M2, M3, M4, or M5 S2 $\rightarrow$ prediction may fail or produce partial results S3 $\rightarrow$ no prediction possible
Model Prediction $\rightarrow$ Tool Selection	M1 $\rightarrow$ T1 M2 $\rightarrow$ T2 M3 $\rightarrow$ duplicates removed, then T1 or T2 M4 $\rightarrow$ invalid tools filtered M5 $\rightarrow$ removed from the final tool list
Tool Selection $\rightarrow$ Execution	T1, T2 $\rightarrow$ SmartBugs execution pipeline is invoked T3 $\rightarrow$ no task is generated or fallback behavior is applied
Configuration $\rightarrow$ Execution Mode	G1 $\rightarrow$ E1 can be configured correctly G2 $\rightarrow$ E1 may use default values G3 $\rightarrow$ configuration error is reported
Execution Mode	E1 $\rightarrow$ intelligent workflow is executed E2 $\rightarrow$ original manual workflow is executed

Table 2: Constraints for the intelligent workflow

10.5 Derived Test Frames

Based on the categories and constraints, the following representative test frames are derived. The detailed test cases corresponding to these frames are specified in the *Test Case Specification* document.

Test Frame ID	Category Combination	Expected Behavior
TF-POST-01	C1, F1	Features are extracted successfully from a valid Solidity contract.
TF-POST-02	C2, F3	Feature extraction fails and the error is reported.
TF-POST-03	F1, S1	The required subset of model features is correctly selected.
TF-POST-04	S1, M1, T1	A single valid tool is predicted and selected.
TF-POST-05	S1, M2, T2	Multiple valid tools are predicted and selected.
TF-POST-06	S1, M3, T1/T2	Duplicate tool predictions are removed.
TF-POST-07	S1, M5, T3	<code>nessun_tool</code> is removed from the final tool list.
TF-POST-08	T1/T2, E1	SmartBugs executes the selected tools using the existing pipeline.
TF-POST-09	E2	The original manual workflow remains available and unchanged.
TF-POST-10	G1, E1	The intelligent workflow command-line options are parsed correctly and stored in the runtime configuration.

Table 3: Derived test frames for the intelligent workflow

11 Conclusion

This test plan defines the strategy adopted to validate the evolution of SmartBugs.

The testing activity will validate both the newly introduced intelligent tool-selection mechanism and the preservation of the original SmartBugs behavior. Detailed test case specifications and execution reports are provided in separate documents.